

Architectural Evolution of Large File Upload Infrastructure

Executive Summary

Adopting Uppy and the TUS protocol provides benefits beyond resumable uploads.

The primary architectural advantage is preserving future deployment options while avoiding a proprietary upload protocol. Initial implementations may use a monolithic architecture where the business application directly receives uploaded files. As upload volume and file sizes grow, the upload tier can be separated from the business tier without requiring frontend changes.

Phase 1 - Initial Deployment

The application directly receives uploaded files.

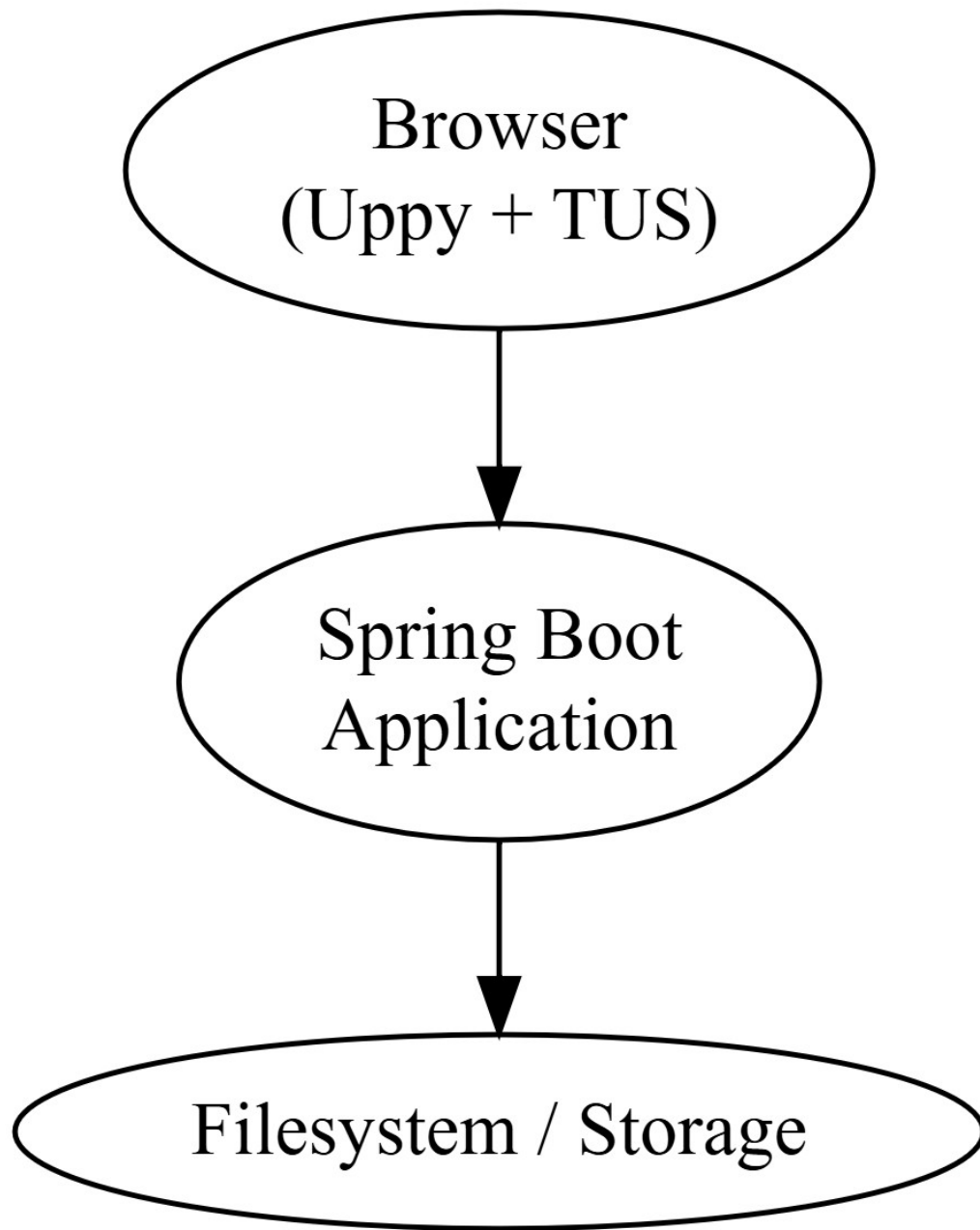
Characteristics

- Simple deployment
- Minimal infrastructure
- Suitable for moderate upload volume
- Business application owns upload lifecycle

Responsibilities of the Business Application

- Authentication
- Upload reception
- Chunk assembly
- Resumability state
- Temporary storage
- Validation
- Business processing

Diagram



Phase 2 - Growth

As file sizes and upload volume increase, the application begins spending substantial resources on upload management rather than business processing.

Typical Growth Drivers

- 100 MB uploads become 1 GB uploads
- 1 GB uploads become 10+ GB uploads

- Increased user concurrency
- Increased storage requirements
- Longer upload durations

Emerging Challenges

- Network I/O saturation
- Memory pressure
- Temporary storage management
- Retry handling
- Upload state persistence
- Horizontal scalability

At this stage the upload subsystem becomes an infrastructure concern rather than a business concern.

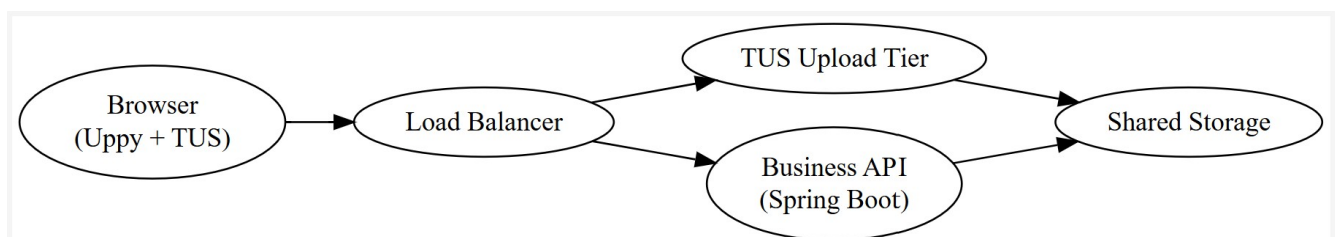
Phase 3 - Dedicated Upload Tier

Upload processing is separated from business processing.

Characteristics

- Dedicated upload infrastructure
- Independent scaling
- Reduced load on business services
- Improved operational flexibility

Diagram



Workflow

1. Browser uploads file to TUS endpoint.
2. Upload tier stores data.
3. Upload completion event is generated.
4. Business application receives metadata.
5. Business processing begins.

Example completion notification:

```
{
  "uploadId": "12345",
  "fileName": "archive.zip",
  "size": 1234567890,
  "location": "/uploads/12345"
}
```

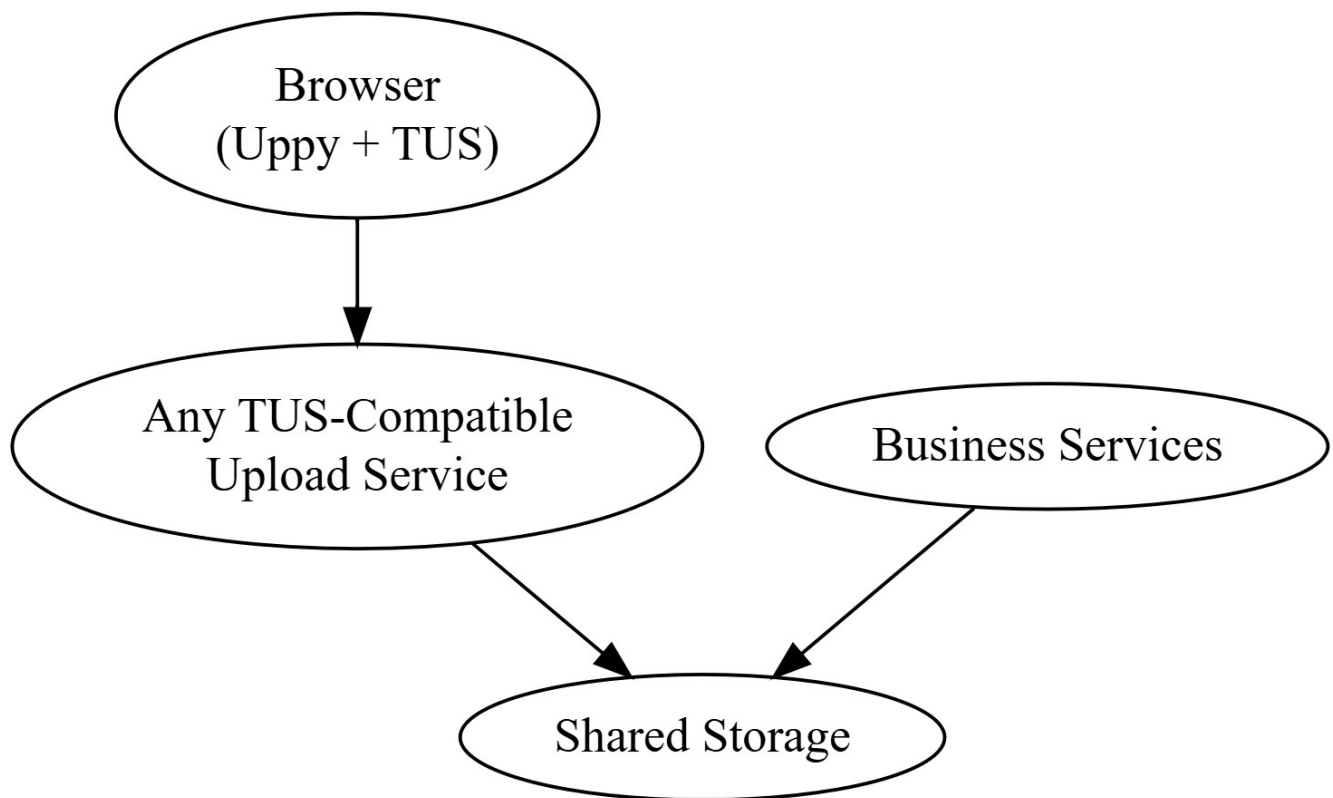
Phase 4 - Infrastructure Optimization

The upload tier may be replaced without changing browser behavior.

Possible implementations include:

- Embedded Java TUS server
- Dedicated Java TUS cluster
- tusd (Go reference implementation)
- ASP.NET TUS server
- Cloud storage upload gateway
- Managed upload service

Diagram



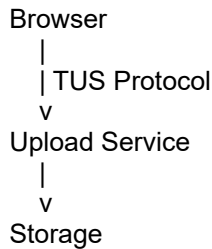
Strategic Benefit of TUS

The primary value of TUS is not resumable uploads alone.

The primary value is establishing a stable protocol boundary between:

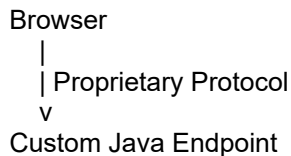
- Upload infrastructure
- Business applications

TUS-Based Architecture



Any TUS-compliant server can potentially replace another implementation with minimal impact on the frontend.

Custom Upload Protocol



The frontend and backend become tightly coupled.

Future migration to:

- tusd
- cloud-native upload services
- dedicated upload appliances
- alternate technology stacks

typically requires protocol redesign and frontend changes.

Long-Term Architectural Conclusion

Adopting Uppy and TUS today preserves future architectural options.

A project may evolve through:

1. Monolithic Java upload handling
2. Dedicated upload subsystem
3. Horizontally scaled upload infrastructure

4. Specialized upload middleware
5. Cloud-native storage gateways

while maintaining a consistent browser-side integration.

This reduces long-term technical debt and avoids ownership of a proprietary upload protocol.